As you can see, a Ruby script has the file extension *rb*. If there is a syntax mistake, the *ruby –c* command will highlight the syntax error, line by line. Complete Ruby programming tutorial videos by Wild Academy are published online in 37 modules in YouTube at *https://www.youtube.com/playlist?list=PLMK2xMz5H5Zv8eC8b4K6tMaE1-Z9FgSOp*

Chef Solo

Chef Solo is a standalone tool, which can be used to test Chef scripts without a Chef container. For example, without installing a Chef server-agent, we can write Chef recipes in a standalone environment and run them using Chef Solo. For this, Chef-apply, which is a wrapper tool built on top of Chef Solo, helps in running Chef recipes without a Chef server set-up. Sample code *test1.rb* is explained below as an example:

```
[root@magdesklinux ~]# cat test1.rb
file 'file.txt' do
content 'This is a test program'
end
```

This program creates a file *file.txt* in the user's directory and writes the content command as a text in the file. Chef-apply shows the output, which is a detailed execution of the script.

```
[root@magdesklinux ~]#  chef-apply test1.rb
Recipe: (chef-apply cookbook)::(chef-apply recipe)
  * file[file.txt] action create
    - create new file file.txt
    - update content in file file.txt from none to 31b31c
        --- file.txt   2015-11-07 02:28:10.000000000 +0530
        +++ /root/magesh/chef/hello.txt20140711-2344-mf17rh
        @@ -1 +1,2 @@
             +This is a test program
```

Creating a cookbook

After testing and validating the Ruby script, let's get into the Chef cookbook using Ruby scripts. In a typical Chef set-up, we need to create a cookbook which is a pre-defined structure of Ruby scripts; this can be created using the knife tool as shown below:

```
[root@magdesklinux ~]# knife cookbook create cookbook1
** Creating cookbook cookbook1
** Creating README for cookbook: cookbook1
** Creating CHANGELOG for cookbook: cookbook1
** Creating metadata for cookbook: cookbook1
```

The execution of the knife *cookbook create cookbook1* command creates the cookbook template, which in turn creates a directory tree structure as shown below:

```
|
|--CHANGELOG.md
|--metadata.rb
|--README.md
|--attributes
|--definitions
|--files
|--libraries
|--providers
|--recipes
|--resources
|--templates
```

In the directory tree structure, the recipes directory file is used to keep recipes (also known as installation scripts), which help in installing and configuration application package deployment. The resources directory is used to store the files required for the installation explained above, such as *readme* files, etc.

There is a pre-defined construct for the Chef script, which is explained in detail here. First, we need to create a package that will be invoked during the installation process. It can be the installation of the code package and/or the configuration package for the required set-up, or an application or service that is required to run as background services.

```
package 'def-package' do
action :install
end

service 'service1' do
action [ :enable, :start ]
end
```

After preparing the package and service, we can write scripts to install/copy the application package or libraries in the recipes folder and then upload the cookbook into the Chef server using the *knife* command, as given below:

```
# knife cookbook upload cookbook1
or
# knife cookbook upload –a
```

The given *-a* option is used to upload all cookbooks as a bulk upload to the Chef server. Once this is done, we need to synchronise the cookbook from the server to all node-agents connected to the Chef server by simply running the commands below in all connected nodes.

```
[root@magdesklinux ~]# sudo chef-client
Starting Chef Client, version 11.8.2
resolving cookbooks for run list: ["cookbook1"]
Synchronizing Cookbooks:
  - apt
  - cookbook1
Compiling Cookbooks...
```